

Zadatak 3

GPU programiranje

CPU = AMD Ryzen 5 6600HS Creator Edition

GPU = AMD Radeon 680M (rev 03)

Zadatak 3.1

Za prebrojavanje prim brojeva u zadanom ulaznom nizu cjelobrojnih vrijednosti napisani su sekvencijalni algoritam u C-u i GPU paralelni algoritam u C-u koristeći OpenCL.

Argumenti koje programi primaju su sljedeći:

- k - određuje broj elemenata $N=2^k$
- (paralelna inačica) atomic t/f - određuje je li operacija dodavanja u konačni zbroj atomarna
- rand/seq - određuje generiraju li se brojevi nasumično ili su sekvencijalni 1..N
- (opcionarno) seed - za postizanje determinizma nasumične generacije u svrhe usporedbe i provjere točnosti

U paralelnoj inačici je još moguće konfigurirati broj blokova i veličina blokova preko makroa.

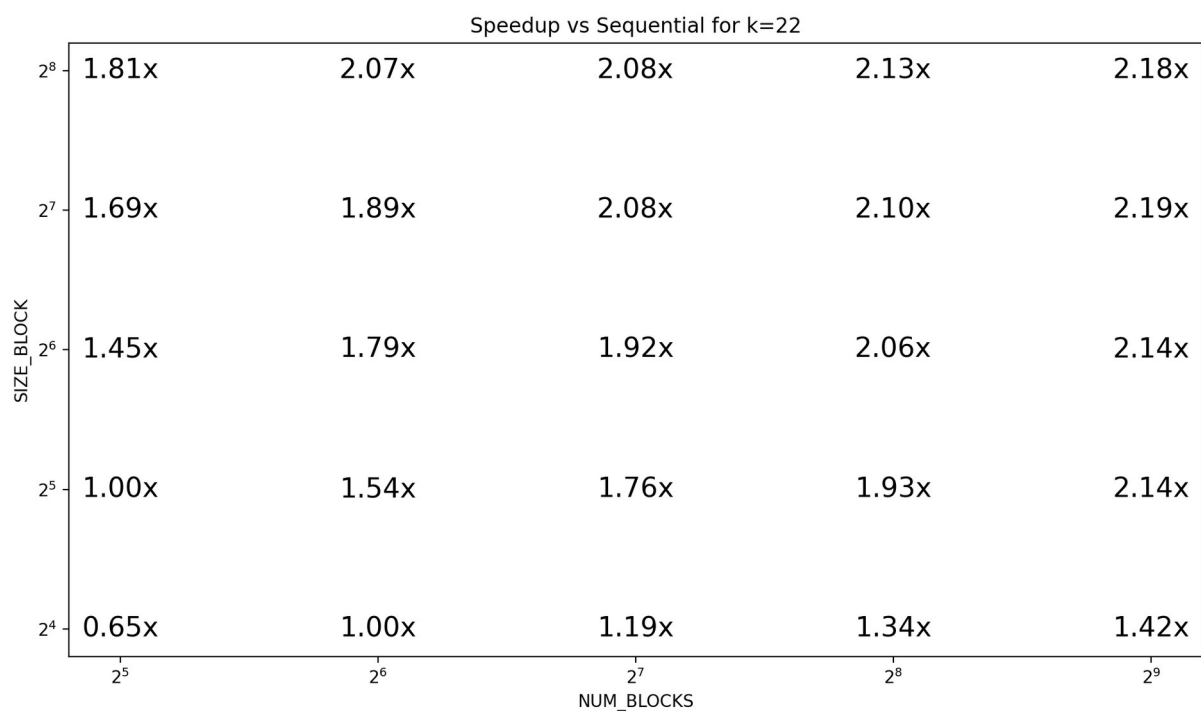
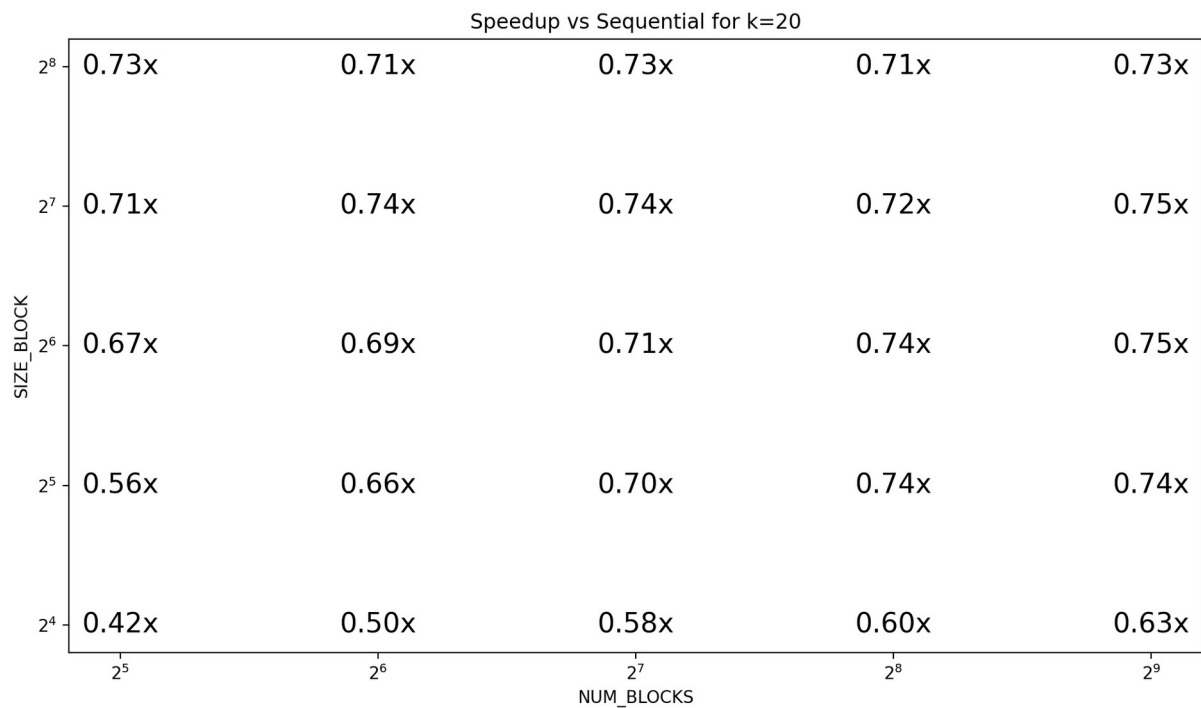
Mjerenja su izvedena za sekvencijalni način generiranja brojeva i za sve kombinacije sljedećih postavki po jednom:

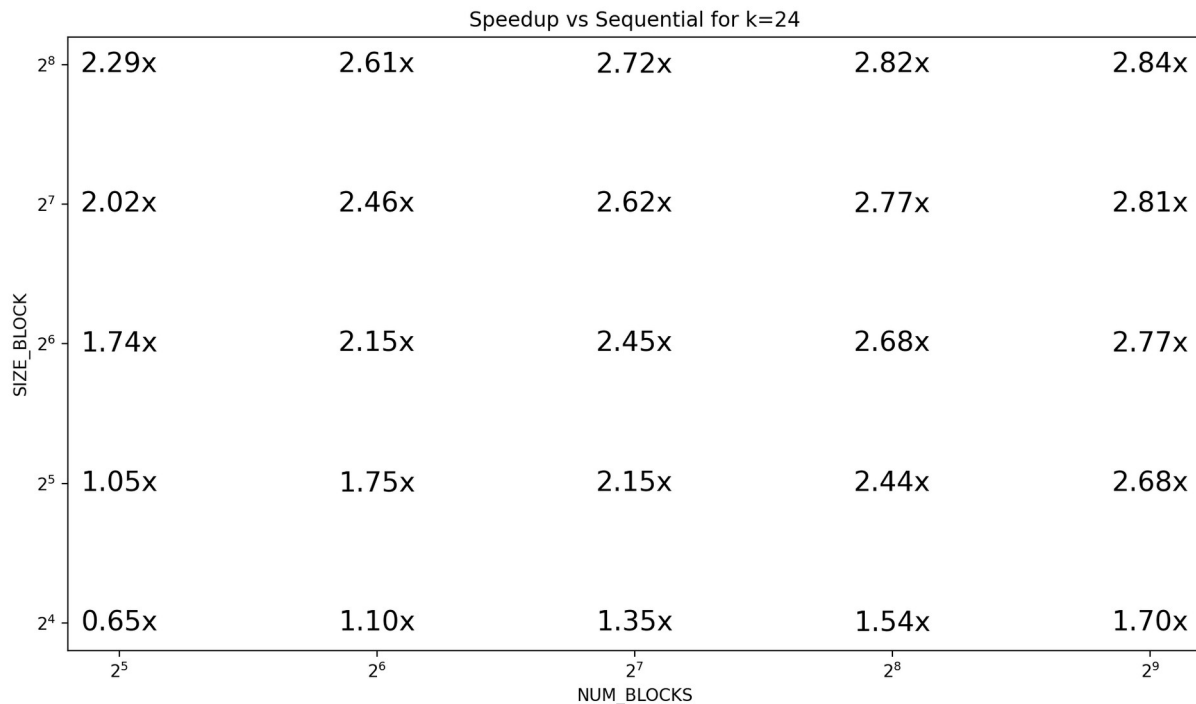
- $k = 20, 22, 24$
- `NUM_BLOCKS` = 32, 64, 128, 256, 512
- `SIZE_BLOCK` = 16, 32, 64, 128, 256
- atomic = t/f

Naravno, za sekvencijalni program varira samo k .

Mjerenja s atomic postavljenim na false daju netočan rezultat za broj prim brojeva u nizu te izvođenje traje otprilike jednako jer je beznačajnog trajanja u usporedbi s izračunom.

Usporedba paralelne inačice za različite postavke blokova sa sekvencijalnom inačicom i s atomarnom operacijom dodavanja za sve tri isprobane vrijednosti k su vidljive u sljedeća tri grafa.





Vidimo da se za $k=20$ sekvencijalni program izvodi brže od paralelnog i to je očekivano jer za tako male količine posla, vrijeme koje se troši na prijenos podataka i pripremanje GPU-u prevladavaju. Ova razlika performansi se smanjuje povećanjem broja blokova i veličine bloka što je također očekivano jer svaka dretva ima ukupno manje posla, a dijelovi posla su u većim komadima.

Za $k=22$ i $k=22$ paralelni program dobiva prednost nad sekvencijalnim jer se vrijeme pripreme GPU-a ne skalira proporcionalno vremenu računanja. Povećanje broja blokova i veličine blokova donosi padajuće poraste u performansama za koje bi očekivali da eventualno krenu biti smanjenja. Dodatni blokovi eventualno samo čekaju, potrebno je više sinkronizacije te memorijska propusnost postaje usko grlo.

Očekivalo bi se da su broj i veličina blokova za koje se postižu optimalne performanse ovisni o N , ali to je teško provjeriti na ovim mjerenjima zbog malih razlika u performansama u blizini optimalne vrijednosti.

Zadatak 3.2

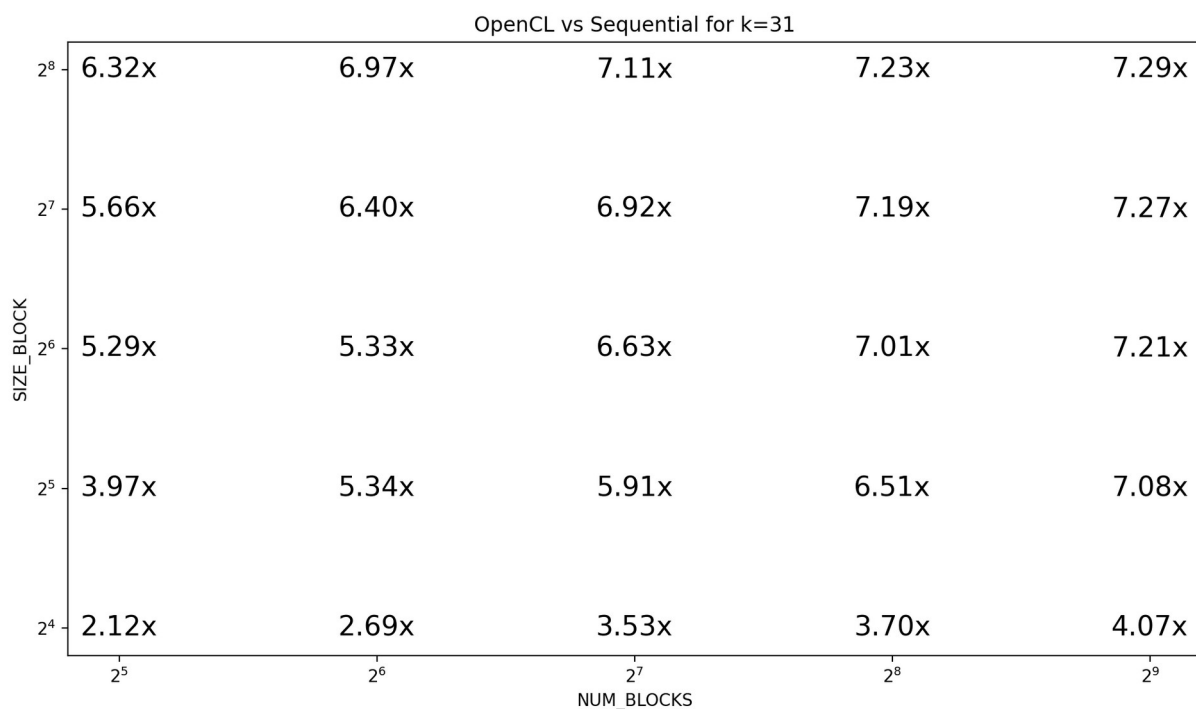
Zadatak aproksimacije broja pi sumom reda izvedena je sekvencijalno u C-u, koristeći OpenCL te koristeći Numba `@vectorize` i `@guvectorize`. Ulazni parametar je N - broj elemenata niza. Završna zbrajanja parcijalnih suma obavljena su sekvencijalno.

Vlastito računalo nema mogućnost pokretanja CUDA kernela, niti CUDA načina rada u Numbi. Zbog toga su Numba programi pokrenuti u Google Colab-u koristeći T4 GPU i međusobno uspoređeni, a usporedbu sa sekvencijalnom i OpenCL inačicom nije imalo smisla

raditi. Postojala je mogućnost pisanja CUDA kernel programa u Google Colab-u, ali je bilo prekasno kada sam postao svjestan ovih problema.

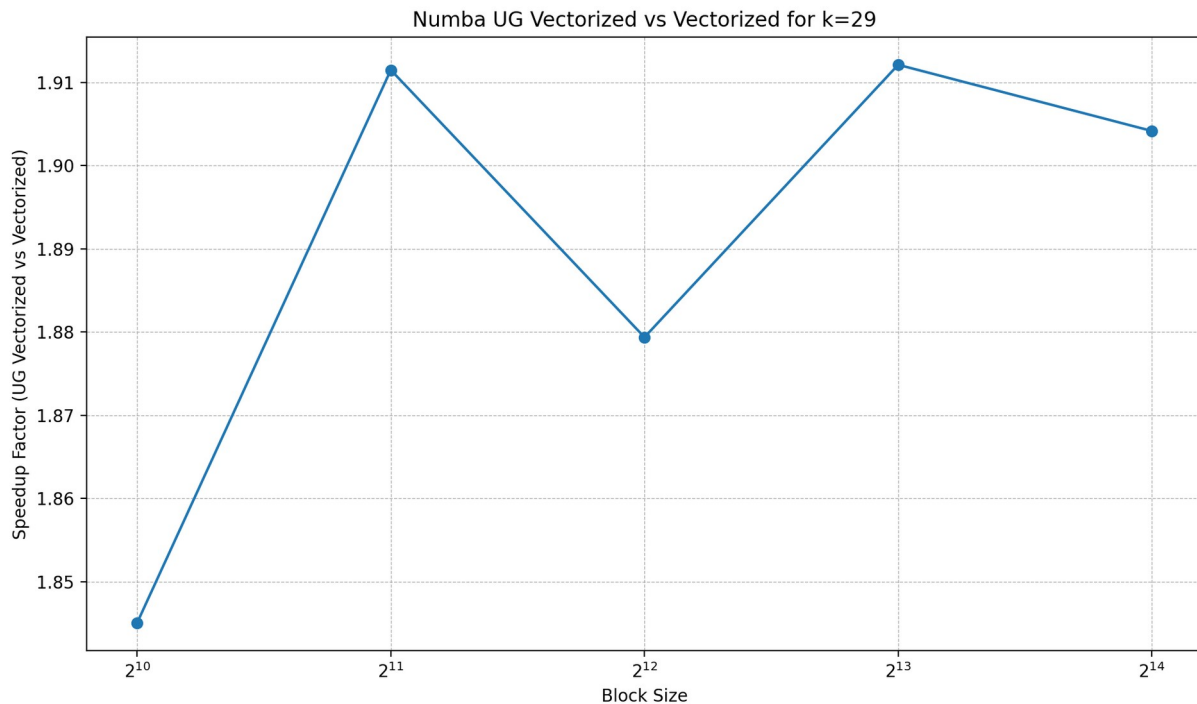
Osim ovih problema, nije bilo moguće raditi s double tipom podataka, već samo s float. Zbog ovoga nije bilo moguće dobiti točne rezultate aproksimacije za velike N-ove, ali su s njima ipak odrađena mjerenja jer i dalje pokazuju dostojna vremena izvođenja za problem.

Izraz za aproksimaciju je promjenjen kako bi se optimiziralo izvođenje na GPU, odnosno smanjen je broj izračuna, a osobito dijeljenja. Na sljedećem grafu su prikazana ubrzanja OpenCL inačice programa naspram sekvencijalne inačice u C-u za $N=2^{31}$



Vidimo da performanse rastu povećanjem broja blokova i veličine bloka.

Sljedeći graf prikazuje ubrzanje @ugvectorized inačice naspram @vectorized Numba inačice programa za $N=2^{29}$ pri različitim veličinama blokova. Uzeti u obzir da y os grafa ne počinje na nula.



Uočavamo da je ubrzanje kod korištenja `@ugvectorized` skoro pa dvostruko.

Zadatak 3.3

Za ovaj zadatak sam izmjenio dobiveni C++ program za simulaciju dinamike fluida u 2D prostoru tako da se izvršava paralelno koristeći OpenCL. Također je bilo potrebno izmijeniti oba programa da rade s float umjesto s double tipom podataka zbog nekompatibilnosti mog hardwarea.

Paralelizirana je funkcija jacobi te je izbjegnuto kopiranje podataka tako da se jednostavnu izmjenjuju dva međuspremnik, onaj koji se koristi za računanje i onaj koji se koristi za spremanje rezultata trenutne iteracije.

Za `scale=64` i `numiter=5000` izmjereno je sljedeće:

- Sekvencijalna izvedba - 18.7075 sekundi
- Paralelna izvedba - 5.38331 sekundi
- Ubrzanje - 3.475x